Trabalho 01: Busca Não Informada, Busca Informada e Busca Local

Pedro Henrique Galvão[1], **João Paulo Cruz de Faria**[1]

[1]Curso de Engenharia de Computação, CEFET-MG, campus Divinópolis
Disciplina de Inteligência Artificial, 2026/2

Abstract. *This report presents the implementation and experimental evaluation of search algorithms applied to two classic artificial intelligence problems: route finding in a weighted grid and the 8-Queens problem. We compared uninformed methods (BFS, DFS, and UCS), informed methods (Greedy Best-First Search and A*), and local search strategies (Hill Climbing and Random-Restart Hill Climbing). The results show that BFS and Greedy Search prioritize shorter paths in number of steps, but cross high-cost terrains. UCS and A* find the optimal path with cost 29, but A* expands fewer states. For the 8-Queens problem, basic Hill Climbing achieved only 12% success due to local minima, whereas Random Restart reached 100% success within the established limit.*

Resumo. *Este relatório apresenta a implementação e a avaliação experimental de algoritmos de busca aplicados a dois problemas clássicos de inteligência artificial: navegação em grade com custos e o problema das 8 Rainhas. Foram comparados métodos não informados (BFS, DFS e UCS), informados (Busca Gulosa e A*) e de busca local (Hill Climbing e Random-Restart Hill Climbing). Os resultados mostram que BFS e Busca Gulosa priorizam rotas com menos passos, mas atravessam terrenos caros. UCS e A* encontram a rota de custo ótimo 29, porém o A* reduz as expansões de 74 para 43 nós com a heurística Manhattan. No problema das 8 Rainhas, o Hill Climbing obteve apenas 12% de sucesso por ficar preso em mínimos locais, enquanto o Random Restart atingiu 100% de sucesso.*

1. Introdução

O objetivo deste trabalho é implementar, comparar e analisar o comportamento de diferentes algoritmos de busca clássicos. O estudo divide-se em duas partes complementares: a navegação de um agente em uma grade com terrenos ponderados e a resolução do problema das 8 Rainhas usando busca local.

Na primeira parte, analisamos como o conhecimento de custos de terreno e estimativas heurísticas influenciam a quantidade de nós expandidos e a qualidade do caminho encontrado. Na segunda parte, investigamos a capacidade do algoritmo Hill Climbing em encontrar configurações sem ataques mútuos entre rainhas, evidenciando suas limitações em terrenos com mínimos locais e a eficácia da estratégia de reinícios aleatórios.

2. Representação e implementação

2.1. Parte 1: Busca não informada e informada

O ambiente da grade retangular foi modelado a partir da leitura de arquivos de texto. Cada estado é representado pelas coordenadas cartesianas $(linha, coluna)$. Os movimentos

permitidos são restritos a quatro direções ortogonais (cima, baixo, esquerda e direita). O custo de uma ação corresponde ao peso da célula de destino: 1 para terreno normal, 3 para terreno irregular e 6 para terreno de alto custo.

A fronteira de busca foi estruturada de acordo com cada algoritmo: uma fila FIFO (usando `collections.deque`) para o BFS, uma pilha (usando lista padrão) para o DFS, e uma fila de prioridade com min-heap (`heapq`) para UCS, Busca Gulosa e A*. Para desempatar nós com mesma prioridade na fila, foi utilizado um contador sequencial de inserção, garantindo a ordem de geração. O controle de estados visitados utilizou conjuntos para busca em grafo, e a reconstrução do trajeto final foi feita a partir de referências ao nó pai de cada estado até o ponto inicial.

2.2. Heurísticas

Foram implementadas duas funções heurísticas: a distância Manhattan e a distância Euclidiana até o estado objetivo. A distância Manhattan soma as diferenças absolutas das coordenadas horizontais e verticais, representando com precisão o deslocamento mínimo em malhas ortogonais. A distância Euclidiana calcula a reta direta entre dois pontos no plano. Ambas foram utilizadas como estimativas de custo restante para orientar a Busca Gulosa e o algoritmo A*.

2.3. Problema das 8 rainhas

Uma configuração do tabuleiro 8×8 foi representada por um vetor de 8 inteiros, onde o índice indica a coluna e o valor armazenado representa a linha em que a rainha está posicionada. Esta representação garante que nunca haverá duas rainhas na mesma coluna.

A função de avaliação $h(s)$ calcula a quantidade de pares não ordenados de rainhas que se atacam mutuamente na mesma linha ou nas diagonais. A vizinhança de um estado possui 56 configurações possíveis, obtidas ao mover cada rainha para qualquer uma das outras 7 linhas de sua coluna. A geração dos vizinhos segue a ordem obrigatória da coluna 0 a 7 e, para cada coluna, das linhas 0 a 7 pulando a linha atual.

No Hill Climbing básico, avaliam-se todos os 56 vizinhos e escolhe-se o primeiro melhor vizinho encontrado. O movimento só é executado se houver melhora estrita ($h_{novo} < h_{atual}$). Não são permitidos movimentos laterais. No Random-Restart Hill Climbing, o método básico é executado repetidamente com novos estados iniciais gerados pelo mesmo gerador pseudoaleatório até encontrar uma solução ($h = 0$) ou atingir o limite de 50 reinícios.

3. Metodologia experimental

Os testes foram realizados em ambiente Windows com Python 3.14. Na Parte 1, os algoritmos foram executados no mapa de teste fornecido (`data/mapa_teste.txt`). Cada configuração foi repetida 30 vezes para obter a mediana do tempo de execução em milissegundos, desconsiderando operações de entrada e saída.

Na Parte 2, tanto o Hill Climbing básico quanto o Random Restart foram executados para as 100 sementes inteiras no intervalo de 0 a 99. Registraram-se a taxa de sucesso, o total de iterações, a quantidade de estados avaliados e os reinícios necessários.

O código completo do projeto e as instruções de execução estão disponíveis no repositório `https://github.com/joaopaulocruzdefaria/trabalho1-ia-search-algorithms` (tag `entrega-final`).

4. Resultados

4.1. Parte 1: Busca não informada e informada

A Tabela 1 resume o desempenho obtido no mapa de teste público para os métodos de busca não informada e informada.

Tabela 1. Resultados dos algoritmos de busca no mapa de teste.

Algoritmo	Heurística	Passos	Custo	Gerados	Expandidos	Pico fronteira	Memória	Tempo (ms)
BFS	-	15	40	60	52	9	60	0.2071
DFS	-	15	40	21	15	6	21	0.0737
UCS	-	29	29	77	74	6	77	0.3712
Gulosa	Manhattan	15	40	21	15	6	21	0.1243
Gulosa	Euclidiana	15	40	21	15	6	21	0.0880
A*	Manhattan	29	29	60	43	15	58	0.2718
A*	Euclidiana	29	29	72	60	12	72	0.3749

A Figura 1 apresenta a comparação visual entre a rota encontrada pelo BFS (priorizando poucos passos) e a rota encontrada pelo A* (priorizando o menor custo acumulado).

Caminho BFS (Passos: 15, Custo: 40)

```
S**************G
.#.#####.####...
.#.....#....#...
.#####.#.##.#...
.....#.#....#...
####.#.######...
.....#...R.R....
................
```

Caminho A* (Passos: 29, Custo: 29)

```
S**MMMMM.....**G
.#*#####.####*..
.#*****#....#*..
.#####*#.##.#*..
.....#*#....#*..
####.#*######*..
.....#***R.R**..
........*****...
```

Figura 1. Comparação dos caminhos no mapa de teste. Asteriscos representam as células visitadas entre o início (S) e o objetivo (G).

4.2. Parte 2: Busca local e 8 rainhas

A Tabela 2 apresenta os resultados dos experimentos com as sementes de 0 a 99 para ambos os métodos de busca local.

Tabela 2. Resultados dos métodos de busca local no problema das 8 rainhas.

Métrica	Hill Climbing	Random Restart
Execuções	100	100
Soluções encontradas	12	100
Taxa de sucesso (%)	12,0%	100,0%
Média de iterações	3,28	24,72
Mediana de iterações	3,0	17,5
Média de estados avaliados	233,96	1770,64
Média de reinícios	N/A	6,76
Máximo de reinícios	N/A	42

Análise de falha do Hill Climbing (Semente 0): Na execução com a semente 0, o algoritmo partiu do estado inicial $[6, 6, 0, 4, 7, 6, 4, 7]$ com 9 conflitos e avançou por 4 iterações consecutivas: $9 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2$. Ao alcançar o estado final $[6, 3, 5, 2, 1, 6, 4, 7]$ com $h = 2$, todos os 56 vizinhos possíveis foram gerados e avaliados. O resultado mostrou que 3 vizinhos mantinham $h = 2$ (empate de valor) e 53 vizinhos apresentavam conflitos maiores ($h > 2$). Como o algoritmo exige melhora estrita e não admite movimentos laterais, a busca foi interrompida em um mínimo local com 281 estados avaliados.

5. Discussão

5.1. Busca Não Informada e Informada (Questões 1 a 7)

Questão 1: Número de ações vs. custo da solução. Sim, o BFS encontrou o menor número de ações, totalizando 15 passos, empatado com DFS e Busca Gulosa. No entanto, este caminho não foi o de menor custo, resultando em um custo final de 40. Isso ocorre porque o BFS prioriza a menor profundidade na árvore de busca, considerando apenas o número de transições e ignorando os pesos do mapa. Em um ambiente com terrenos heterogêneos, a rota com menos passos acabou atravessando células de custo elevado, enquanto algoritmos que consideram custo, como UCS e A*, encontraram uma rota alternativa de 29 passos com custo ótimo de 29.

Questão 2: Divergência entre BFS e UCS em ambientes ponderados. A divergência decorre do critério de ordenação da fronteira. O BFS utiliza uma fila FIFO e expande os nós por nível de profundidade, garantindo o caminho com menos arestas independentemente do custo individual de cada passo. Em contrapartida, o UCS utiliza uma fila de prioridade ordenada pelo custo acumulado $g(n)$, expandindo sempre o nó de menor custo de caminho. Assim, quando o trajeto mais direto passa por terrenos caros, o BFS opta por essa rota curta e cara, enquanto o UCS contorna os obstáculos por caminhos que exigem mais passos, mas somam menor custo total.

Questão 3: Influência da ordem de sucessores na DFS. O DFS opera com pilha e explora exaustivamente o primeiro ramo descendente até o fim antes de realizar backtracking. Como não possui informação de custo nem orientação heurística, a ordem de geração dos sucessores determina diretamente qual região do espaço de busca será explorada primeiro. No mapa de teste, a ordem dos sucessores coincidiu com a direção do objetivo, permitindo ao DFS encontrar a solução com apenas 15 expansões por puro fator de sorte. Caso a ordem fosse desfavorável, o DFS teria se aprofundado em ramos infrutíferos, expandindo muitos nós desnecessários e retornando um caminho excessivamente longo.

Questão 4: Poucas expansões e custo elevado na Busca Gulosa. A Busca Gulosa avalia a fronteira exclusivamente pelo valor da heurística $h(n)$, que estima a distância restante até o objetivo, desconsiderando o custo acumulado $g(n)$ do caminho. Guiado apenas pela proximidade geográfica da meta, o algoritmo avança em linha reta e conclui a busca com apenas 15 estados expandidos. Todavia, ao ignorar o custo real das transições, o algoritmo atravessou células de terreno difícil com custo final de 40 em vez de realizar desvios por caminhos mais baratos.

Questão 5: Comparação entre UCS e A* em custo e expansões. Sim, tanto o A* quanto o UCS convergiram para a solução de custo ótimo de 29. Essa equivalência é garantida porque o UCS é ótimo para custos positivos e as heurísticas utilizadas no A* são admissíveis. A diferença central está no esforço computacional: o UCS expandiu 74 nós ao se propagar de maneira uniforme em todas as direções, enquanto o A* guiado pela heurística Manhattan expandiu 43 nós. A função combinada $f(n) = g(n)+h(n)$ orientou a busca diretamente na direção da meta, podando estados irrelevantes.

Questão 6: Efeito da troca de Manhattan por Euclidiana. Na Busca Gulosa, ambas as heurísticas produziram o mesmo caminho de 15 passos e custo 40 com 15 expansões. No A*, ambas alcançaram o custo ótimo 29, mas a heurística Manhattan foi mais eficiente, expandindo 43 nós contra 60 da Euclidiana. Como os movimentos são apenas nas quatro direções cardeais, Manhattan reflete a geometria real da grade e fornece estimativas mais precisas, enquanto a distância Euclidiana subestima mais os passos restantes e amplia o espaço explorado.

Questão 7: Admissibilidade das heurísticas. Sim, ambas as heurísticas são admissíveis. Uma heurística é admissível quando nunca supera o custo real até o destino. Como o menor custo por passo no mapa é 1 e os movimentos são exclusivamente ortogonais, o menor custo real para percorrer Δx e Δy é no mínimo $\Delta x + \Delta y$. Como a distância Euclidiana é menor ou igual à Manhattan e ambas são menores ou iguais ao custo real, nenhuma delas superestima o caminho.

5.2. Busca Local e 8 Rainhas (Questões 8 a 11)

Questão 8: Hill Climbing vs. Busca Gulosa. Apesar de ambos utilizarem uma função de avaliação heurística, eles possuem estruturas de operação distintas. A Busca Gulosa mantém uma fronteira com nós alternativos na memória, permitindo retornar e explorar outros ramos se o caminho atual parecer ruim. O Hill Climbing opera em busca local sem manter fronteira nem árvore, guardando apenas o estado corrente. Suas decisões são imediatas e sem volta; ao parar em um mínimo local, o algoritmo não possui caminhos guardados para retomar a busca.

Questão 9: Falhas na paisagem de busca do Hill Climbing. A falha do Hill Climbing ocorre pela presença de mínimos locais e ombros na topologia do problema. No caso analisado da semente 0, o algoritmo atingiu um estado com 2 conflitos cercado apenas por vizinhos com 2 ou mais conflitos. Como não são permitidos movimentos laterais nem passos que piorem a avaliação, o algoritmo fica impossibilitado de sair dessa bacia de atração e encerra a execução sem atingir o zero absoluto.

Questão 10: Reinícios aleatórios e suas limitações teóricas. Os reinícios aleatórios começam a busca em diferentes regiões do espaço de configurações, aumentando a probabilidade de iniciar em uma bacia que conduza diretamente à solução global. Nos testes, essa estratégia elevou o sucesso de 12% para 100%. Contudo, o método não altera a miopia do Hill Climbing em cada tentativa individual. Em problemas com espaços

maiores ou se o número de reinícios for restrito, não há garantia absoluta de que a solução será encontrada antes do término das tentativas.

Questão 11: Completude e optimalidade dos algoritmos. O BFS é completo em grafos finitos, mas não garante o menor custo em mapas ponderados; nos experimentos, encontrou o objetivo (15 passos), mas produziu um custo subótimo de 40. O DFS é completo sob controle de ciclos em grafos finitos, mas não é ótimo, também retornando a rota cara de custo 40. O UCS é completo e ótimo para custos de passo estritamente positivos, convergindo para o custo mínimo de 29 ao custo de 74 expansões. A Busca Gulosa não é completa nem ótima, priorizando a proximidade da meta e gerando uma rota cara de custo 40. Por fim, o A* com heurística admissível assegura completude e optimalidade, alcançando o mesmo custo ótimo de 29 do UCS, porém com maior eficiência computacional (apenas 43 expansões com Manhattan).

6. Conclusão

Os experimentos evidenciaram as diferenças práticas entre as abordagens de busca. Na navegação em grade, algoritmos que ignoram os custos do terreno (BFS, DFS e Gulosa) geraram rotas com menos ações, porém de alto custo. O A* com heurística Manhattan mostrou-se a melhor escolha, alcançando o custo ótimo de 29 com apenas 43 expansões. No problema das 8 Rainhas, o Hill Climbing básico sofreu com mínimos locais (12% de sucesso), enquanto o Random Restart alcançou 100% de sucesso, comprovando a eficácia dos reinícios.

Contribuições dos autores

João Paulo Cruz de Faria implementou o BFS, as heurísticas, as buscas por fila de prioridade (UCS e A*), as operações base das 8 Rainhas e o Hill Climbing básico, além de redigir as questões 1 a 5 no relatório. Pedro Henrique Galvão implementou o DFS, a Busca Gulosa, os testes automatizados, o Random-Restart Hill Climbing e o script de experimentos, além de diagramar o relatório SBC, redigir as questões 6 a 11 e gerenciar as entregas da Parte 1 e final.

Declaração de uso de IA

Ferramentas de IA foram utilizadas exclusivamente para revisão textual, esclarecimento de conceitos e apoio na formatação de scripts auxiliares, com verificação dos autores e registro no arquivo `IA_USAGE.md`.

Referências

Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition.